EStream: Fast and Memory-Efficient MoE Prefill through Expert Virtualization on Mobile NPUs

Junming Zhang*
elliott-z@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Zhenzhe Zheng*†
zhenzhezheng@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Fan Wu
wu-fan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

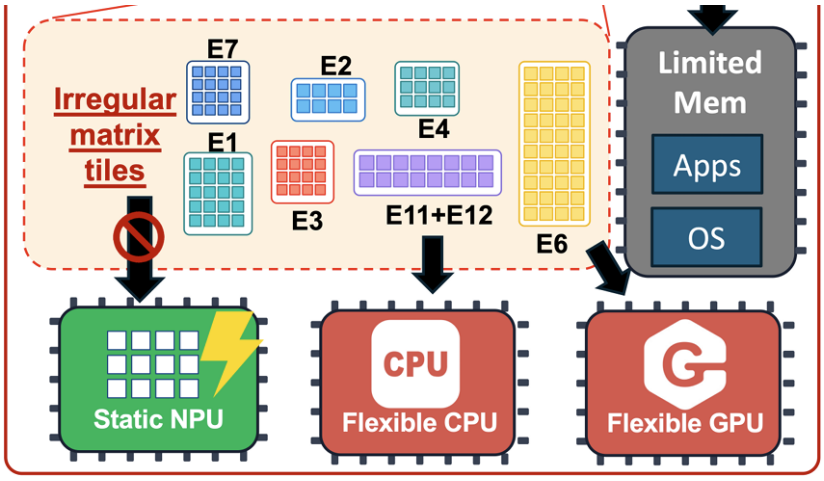
Xiaoyao Huang
huangxy32@chinatelecom.cn
Cloud Computing Research Institute, China Telecom
China

Jie Wu
jiewu@temple.edu
Temple University
Philadelphia, Pennsylvania, USA

一、背景动机

1. 稠密模型 vs MoE 模型

- **原理区别**：稠密模型工作时，每个输入 token 需要与**所有**权重参数进行计算；MoE 模型条工作时，每个输入 token 只与 **Top-K** 的专家权重参数进行计算。
- **稀疏收益**：MoE 核心是**以存代算**，以更高的内存/存储开销，换取更低的计算开销，Qwen3-30B-A3B 在所有测试集上的表现都超过了 Qwen3-4B。
- **与 NPU 的适配程度**：NPU 喜欢**图结构稳定**的稠密模型，而 MoE 模型激活的专家是不规则的（如图所示），与 NPU 配合的不好。有时候只能让 CPU/GPU 去做计算，代价是性能远不如 NPU（4-6 倍的差距）。



补充知识：NPU 运行时的图结构 = 拓扑结构（算子与计算流程，MoE **静态**）+ 张量形状（数据维度，MoE 动态）+ 参数地址（模型权重放在哪块内存，MoE 动态）……

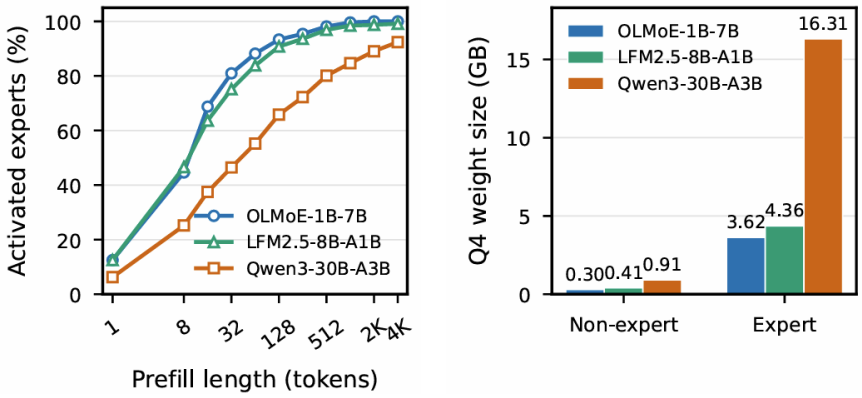
2. 一种重要的端侧 AI 场景：Prefill-only

- **对话场景的 prefill 与 decode**：常见的理解是，输入一串提示词后，模型生成第一个 token 前的过程是 prefill，生成第一个 token 后的过程是 decode。

- **本质的 prefill 与 decode**：更深层次的理解是，prefill 阶段是模型在理解提示词，而 decode 阶段是模型在续写提示词。很多场景只需要模型停止在 prefill 阶段就够了，随后通过一些方法获取模型对这段提示词的基本判断。
- **prefill-only 的例子**：①输入一段电话录音文本，判断是否是诈骗/广告，不需要生成完整回答。②输入“帮我订一张去北京的机票”，捕捉到用户的意图是“订机票”，不需要生成完整回答。

3. 核心问题：端侧 NPU 难以高效执行 MoE Prefill

- **挑战 1**：MoE 模型的图结构在运行时是动态变化的，而 NPU 只能处理在编译时静态确定的图结构，卸载到 CPU/GPU 去处理会非常慢。
- **挑战 2**：虽然每个 token 只激活 Top-K 个专家，但是一段较长的提示词几乎可以激活所有专家（左图），导致内存占用爆炸（右图）。如果没有合适的管理方法，相同性能条件下，MoE 模型的内存压力比稠密模型更大。



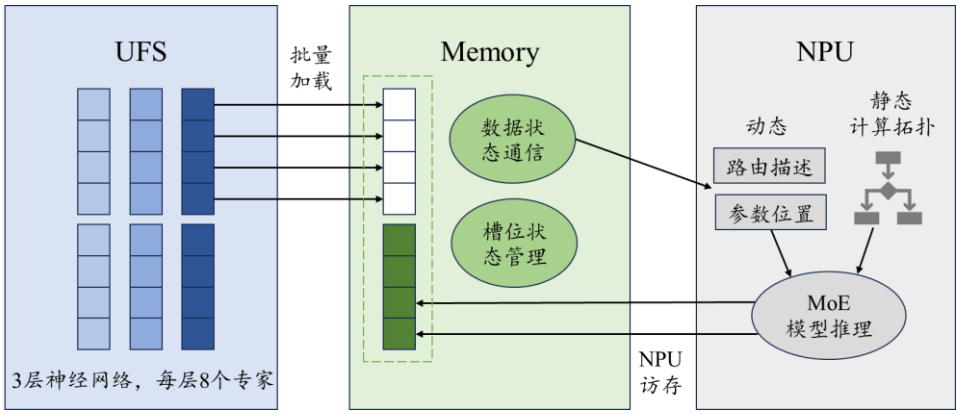
(a) Expert activation **(b)** Q4 weight distribution

二、设计方案

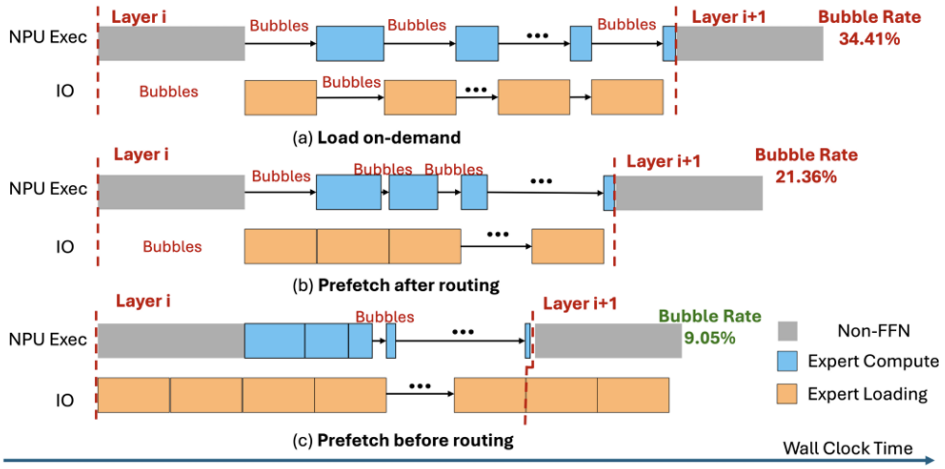
1. 改进机会

- NPU 传统的运行时上下文会把三类信息耦合在一起：拓扑结构（计算相关）、张量形状（数据相关）、参数地址（数据相关）。MoE 条件下，不同专家的**拓扑结构是稳定的**，变化的是张量形状和参数地址，需要把它们解耦。
- MoE 专家具有三个良好的特性：**只读不改、历史状态无关、专家间布局统一**。而且，NPU 一轮计算只会用到一组专家，用完即弃，存在动态驱逐预取机会。
- 随着提示词长度增加，NPU 计算开销同步增加，而 I/O 加载开销存在理论上限（把全部专家加载一遍）。也就是说，在长提示词的条件下，存在**用 NPU 计算掩盖 UFS 加载的流水线机会**。

2. 系统概览



- **NPU 的部分（静态拓扑与动态数据解耦）**：在编译阶段，EStream 将各专家通用的计算流程拓扑图传递给 NPU；在运行阶段，EStream 将路由描述（例如 token 1 要去专家 3，权重是 0.9）和参数地址（专家 3 权重的内存地址是 XXX）动态更新给 NPU。最终实现**一张计算拓扑图服务所有专家**，解决了 MoE 动态路由与 NPU 静态计算不兼容的问题。
- **内存的部分（空间虚拟化）**：以 DMA-BUF 为载体，准备了一个专属内存区域，这个内存区域包括 N 个槽位，每个槽位可以存放一个专家的 gate、up、down 矩阵。维护一个简单的状态机来标识各个槽位的状态：Available、Filling、Ready、InUse。最终实现了 MoE 实际内存占用可控，而 NPU 以为整个 MoE 模型都在内存里。
- **外存的部分（时间流水线）**：MoE 模型的计算过程通常是：收到请求→非专家计算→路由→专家计算→汇总→进入下一层→…。因此，EStream 趁 NPU 执行非专家计算时，盲目批量预取若干专家组进入内存；等路由结果出来后，保留真正需要的专家组，驱逐不需要的专家组；在专家计算阶段，laoder 持续加载后续专家组，NPU 持续执行已就绪的专家组，计算与 I/O 全程重叠。



三、实验结果

平台：一加 15 手机

配置：高通骁龙 8 Elite Gen 5，16GB LPDDR5x，1TB UFS 4.1

- **效果 1（推理加速）**：TTFT 缩短 2.25 到 27.57 倍、长任务耗时缩短 3.79 到 5.75 倍、长提示词条件下相比 MoE 完全驻留在内存只慢 4.3%到 23%。
- **效果 2（内存节省）**：相比非卸载的基线减少 3.73 到 6.36 倍的内存占用，相比 PowerInfer 减少 1.16 到 1.69 倍的内存占用。
- **效果 3（能效提升）**：能效整体提升 1.19 到 8.41 倍，几何平均是 4.08 倍。

效果这么夸张的原因分析

- NPU 的加速效果好于 GPU 和 CPU，而其他方案没有很好地用起来
- 动态加载对比全量加载，肯定有巨大的内存占用优势
- 耗时更短肯定能效更高啊，NPU 不用一直等待了，执行时间缩短

Model	Tokens	Pure TTFT (s) ↓	Peak mem. (GiB) ↓	Tokens/J ↑
Qwen3-30B-A3B	1K	4.838	1.461	43.59
	2K	5.043	1.594	62.36
	3K	5.368	1.729	70.64
	4K	6.836	1.866	72.18
Mixtral-8×7B	1K	7.086	2.767	23.41
	2K	8.742	3.352	29.65
	3K	10.971	3.917	33.17
	4K	13.584	4.114	35.76

支持 30B 激活 3B 的模型（上），峰值内存占用控制在 1.9GB 以内。

支持 46.7B 激活 12.9B 的模型（下），峰值内存占用控制在 4.2GB 以内。

局限性分析

- 对 prefill 阶段有用，对 decode 阶段没用
- 在短提示词（小于等于 80 tokens）的情况下不如直接用 CPU 运行 MoE